

FIIT STU

Dátové štruktúry a algoritmy

Filip Remšík

ID:110883

2020/2021

Správca pamäti

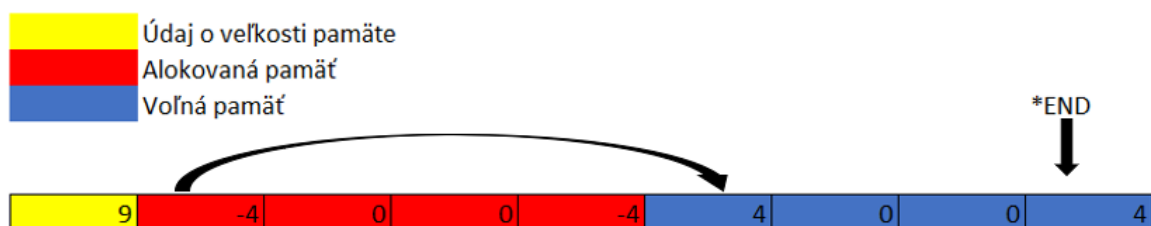
Cvičenie štvrtok 11:00

1. Zadanie úlohy

Zadaním úlohy bolo implementovať vlastnú verziu alokácie pamäti ,pričom sa nesmeli používať existujúce funkcie malloc a free. Implementované funkcie by mali fungovať spôsobom ako tie reálne. Taktiež bolo potrebné otestovať správnu funkčnosť programu a jeho správanie pri rozdielnych hodnotách.

2. Opis môjho algoritmu

Základ môjho programu je metóda implicitných zoznamov. Pri hľadaní voľného bloku cyklus prechádza pamäť od začiatku po koniec pričom skáče z hlavičky na hlavičku a zastaví sa na konci pamäte. Aby sa cyklus nedostal mimo pamäť, je na poslednú päť nastavený pointer, akonáhle ho cyklus dosiahne, zastaví sa.



3. Funkcie

3.1. Memory init

V tejto funkcii si pripravím pamäť na prácu v ďalších funkciách. Do pointeru s názvom memory typu char sa uloží začiatok pamäte. Celú pamäť sa nastaví na hodnotu 0. Na začiatku pamäte je uložená jej celková veľkosť, nasleduje hlavička prvého bloku. Na konci pamäte je päť posledného bloku.

Zložitosť: $O(n)$ -použitie funkcie memset.

Rozloženie mojej pamäte

Celková veľkosť(4B)	Hlavička(4B)	Alokovaná Pamäť	Pätička(4B)
---------------------	--------------	-----------------	-------------

```
void memory_init(void *region, unsigned int size) {
    memory = region;
    memset(memory, 0, size);
    *((int*)(memory)) = size;           //hlavička pamäte
    *(((int*)(memory)) + 1) = size-4;   //začiatok ukladacieho priestoru
    *(END) = size-4;                   //koniec pamäte + posledná päť
}
```

3.2. Memory alloc

Funkciu slúžiaca na vytvorenie miesta v pamäti a následne vracajúca odkaz na začiatok pamäte užívateľa.

Funkcia najskôr na základe porovnania vstupu od užívateľa a veľkosti pamäte overí, či sa oplatí pamäť prehľadávať, ak áno, začne je hľadať miesto pomocou first fit. Oproti metóde best fit je mnou zvolená metóda rýchlejšia ale niekedy zbytočne delí bloky ktoré sú moc

veľké alebo nie je ich možné rozdeliť na menšie. Po nájdení miesta na základe požadovanej pamäte sa blok delí alebo nedelí. V prípade že by mali zostať samostatne 3 voľné bajty, budú pripočítané k pamäti užívateľa. V prípade zostatku 4 bajtov sa vytvorí nová hlavička a petička v jednom, ktorá slúži na lepšie spájanie voľných blokov. Zložitosť: $O(n)$ -pri prehľadávaní celej pamäte, inak je časová zložitosť menšia.

3.3. Memory free

Funkcia slúži na uvoľnenie alokovaného bloku a na spájanie voľných blokov do jedného. Na základe prijatého ukazovateľa sa skontrolujú susedné bloky daného bloku. Podľa toho ktorý z nich je prázdny (hlavička a päta obsahuje kladné číslo), určí ako budú bloky spojené. Ako prvé som si skontroloval, či niektorý zo susedov nie je koniec alebo začiatok, keďže údaj o veľkosti pamäte je kladné číslo a funkcia by ho mohla považovať za voľný blok. Zložitosť: $O(1)$ -porovnávam len veľkosti susedov

```
if (((PREV == memory)&&(NEXT >= END))||((*PREV<0)&&*((int*)(NEXT)<0)) || ((PREV == memory) && ((*int*)(NEXT) < 0))||((*PREV<0) && (NEXT > END))) {
    *((unsigned int*)(valid_ptr)-1)= abs(*((unsigned int*)(valid_ptr)-1));
    *((int*)(NEXT - 4)) = abs(*((int*)(NEXT - 4)));
    return(0);
}
```

3.4. Memory check

Vstupom je platný pointer. Najskôr funkcia preverí či ukazuje do inicializovanej pamäte, až následne pamäť prehľadáva, pričom preveruje či pointer ukazuje na 0 (miesto určené na ukladanie) a či blok v ktorom sa nachádza bol alokovaný(hlavička musí byť zápornej hodnoty).

Zložitosť: $O(n)$ -ak je pointer na konci, prehľadávam celú pamäť.

```
int memory_check(void*ptr){
    char* look = START;
    if( START>= ptr || END < ptr||ptr==NULL){
        return(0);
    }
    while (look < END) {
        if ((ptr>=(look+4))&& (ptr <= (look + abs(*((int*)(look)))-5 ))&&*((int*)(look))<0) {
            return(1);
        }
        look = look + abs(*((int*)(look)));
    }
    return(0);
}
```

4. Testy

Na overenie funkčnosti programu som spravil tester, ktorý alokoval pamäť zadanej veľkosti "range" náhodnou veľkosťou z intervalu <min,max>. Tester podľa ideálneho riešenia „alokuje“ pamäť pokiaľ neostane miesto menšie ako najmenší blok a zároveň alokuje pamäť ale už s hlavičkou a päťou. Ukladá si ktorá alokácia koľko blokov zaplnila. Okrem tohto testera je v projekte ešte jeden main v ktorom sa skúšajú aj ostatné funkcie na jednoduchom príklade.

```
int main() {
    char region[200];
    char* pointer[20];
    memory_init(region, 200);
    int range = 200;
    int sum = 0;
    int min = 8;
    int max = 24;
    int a = -1;
    int alloc=0;
    int pamet = 0;
    while (range > sum + min) {
        int num = (rand() % (max - min + 1)) + min;
        if (sum + num <= range) {
            sum = sum + num;
            a++;
            pointer[a] = memory_alloc(num);
            if (pointer[a]) {
                alloc++;
                pamet = pamet + num+8;
            }
        }
    }
}
```

Výsledky testov s danými hodnotami.

Pamet velkosti 200 s blokmi od 8 bajtov do 24 bajtov.
Vysledky: alokovanych 66.67 % blokov

Pamet velkosti 8000 s blokmi od 500 bajtov do 5000 bajtov.
Vysledky: alokovanych 80.00 % blokov

Pamet velkosti 55000 s blokmi od 8 bajtov do 50000 bajtov.
Vysledky: alokovanych 70.00 % blokov

Pamet velkosti 55000 s blokmi od 8 bajtov do 5000 bajtov.
Vysledky: alokovanych 95.83 % blokov

5.Záver

Síce vytvorený program úplne nesimuluje reálne alokovanie, ale človek aspoň získal prehľad o tom ako to funguje a zlepšil svoje poznatky o pointroch. Asi by som ešte pomenil niektoré zápisy za jednu premennú.